



Image Representation & Digital Image Fundamentals

Learning Objectives

- **Explain** how a physical scene becomes a digital image, and what sampling and quantization mean.
- **Describe** an image as a matrix of pixels, and reason about resolution, bit depth, and intensity.
- **Manipulate** RGB and grayscale representations, and know when to use each.
- **Apply** normalization techniques correctly and explain why they matter for training neural networks.
- **Write Python code** that loads, inspects, transforms, and normalizes MILK10k images.

Today's Agenda

PART 1 — THEORY

80 min

BREAK

20 min

PART 2 — HANDS-ON PYTHON

80 min

- **Part 1 (Theory):** image acquisition, pixels, resolution, RGB & grayscale, histograms, normalization, tensors.
- **Break:** 20 minutes.
- **Part 2 (Practice):** load MILK10k images as arrays, split & visualize channels, convert to grayscale, plot histograms, normalize, and build a full preprocessing function.

Recap — From Session 1

- **Computer Vision** = teaching machines to extract **scene meaning** from images, not just measure light.
- Vision tasks range from **classification** to **detection** to **segmentation** — increasing in precision and cost.
- Our course project, **MILK10k**, is a skin-lesion classification challenge: images + metadata.csv + diagnosis labels.
- **Today:** how an image actually becomes **numbers** a model can compute with — pixels, intensity, resolution, RGB, grayscale, normalization.



SESSION 2

Part 1 — Theory

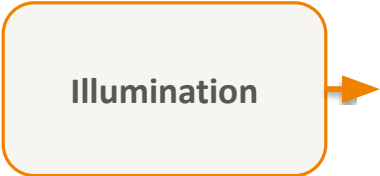
How does an image become numbers?

From Physical Scene to Digital Image

- Image acquisition is the first step in turning a physical phenomenon (what we see) into a digital representation (what a computer sees).

From Physical Scene to Digital Image

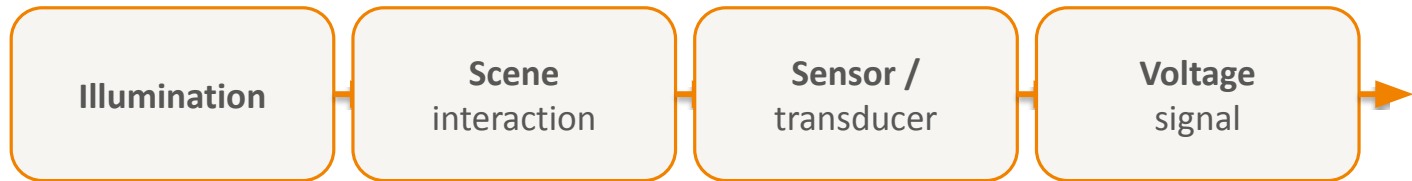
- Image acquisition is the first step in turning a physical phenomenon (what we see) into a digital representation (what a computer sees).
- It starts with an **illumination source** interacting with the subject — ordinary light, but also electromagnetic or ultrasound energy in specialized imaging (e.g., medical scans).



Illumination

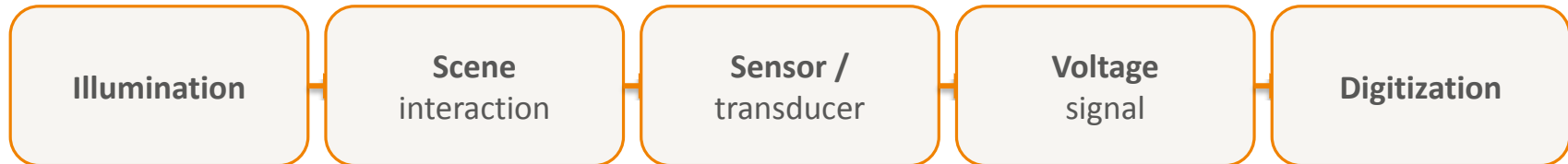
From Physical Scene to Digital Image

- Image acquisition is the first step in turning a physical phenomenon (what we see) into a digital representation (what a computer sees).
- It starts with an **illumination source** interacting with the subject — ordinary light, but also electromagnetic or ultrasound energy in specialized imaging (e.g., medical scans).
- That interaction produces energy **reflected from** or **transmitted through** the scene, captured by a **sensor** — a transducer that converts incident energy into an electrical voltage.



From Physical Scene to Digital Image

- Image acquisition is the first step in turning a physical phenomenon (what we see) into a digital representation (what a computer sees).
- It starts with an **illumination source** interacting with the subject — ordinary light, but also electromagnetic or ultrasound energy in specialized imaging (e.g., medical scans).
- That interaction produces energy **reflected from** or **transmitted through** the scene, captured by a **sensor** — a transducer that converts incident energy into an electrical voltage.
- The voltage signal is then **digitized**, producing the digital image we work with.



Sampling and Quantization

- Turning a **continuous** scene into a **digital** image requires two separate approximations:

Sampling

Quantization

Both are lossy approximations — resolution is the practical result of the sampling choice; color depth is the practical result of quantization choice.

Sampling and Quantization

- Turning a **continuous** scene into a **digital** image requires two separate approximations:

Sampling

- **Sampling** discretizes **space** — deciding how many pixel locations (x, y) to measure.

Quantization

- **Quantization** discretizes **amplitude** — rounding each measured intensity to one of a finite set of levels (e.g., 256 levels for 8-bit).

Both are lossy approximations — resolution is the practical result of the sampling choice; color depth is the practical result of quantization choice.

Sampling and Quantization

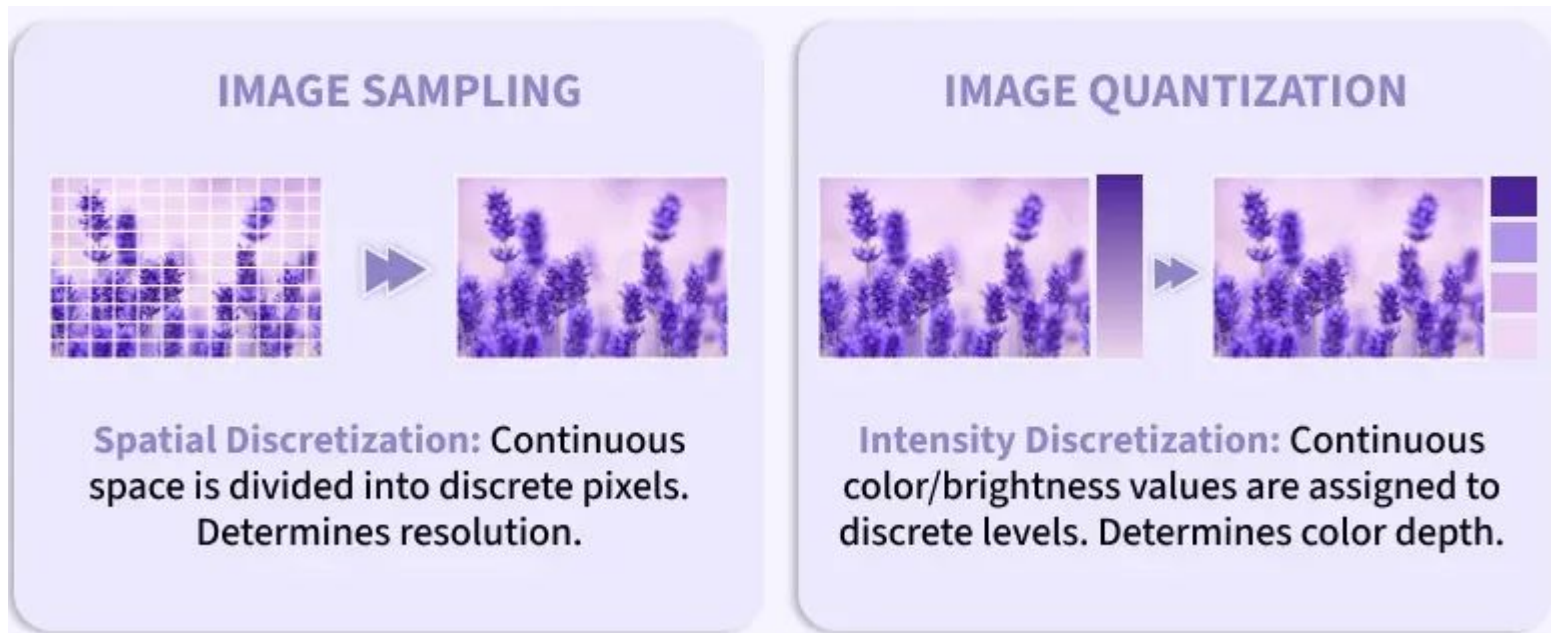
- Turning a **continuous** scene into a **digital** image requires two separate approximations:



Both are lossy approximations — resolution is the practical result of the sampling choice; color depth is the practical result of quantization choice.

Sampling and Quantization

- Turning a **continuous** scene into a **digital** image requires two separate approximations:



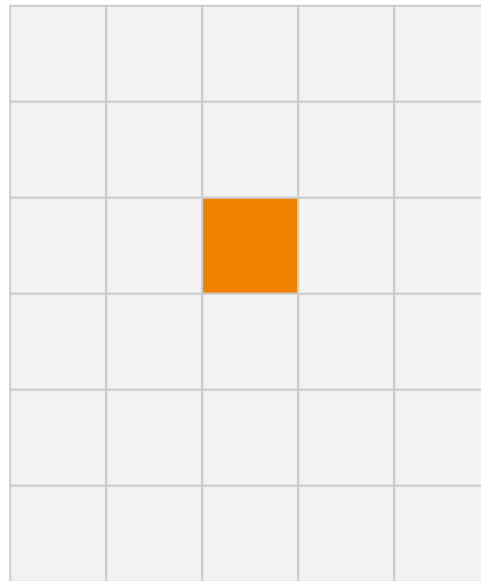
Both are lossy approximations — resolution is the practical result of the sampling choice; color depth is the practical result of quantization choice.

What Is an Image, Formally?

- An image is an **n-dimensional function**. In 2D, we write it **$F(X, Y)$** , where X and Y are spatial coordinates.
- The **amplitude of F** at a coordinate (x, y) is the pixel's **intensity**, or gray level.
- When X , Y , and the amplitude are all finite, we have a **digital image** — a finite grid of pixels a computer can store and process.
- This is the foundation for everything today: pixels, resolution, color channels, and normalization are all just **properties of this function F** .

Pixels as a Matrix

? *If an image is 'just a matrix', what do its row and column actually mean?*

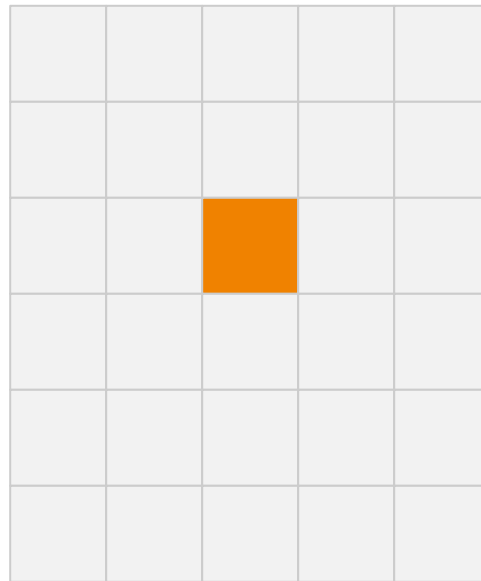


Pixel (row=2, col=2) highlighted

Pixels as a Matrix

? *If an image is 'just a matrix', what do its row and column actually mean?*

- A digital image of size **H × W** is a matrix with **H rows** and **W columns** — each cell is one **pixel**.
- Pixels are indexed as (row, column). In most CV/ML libraries, **(0, 0)** is the **top-left** corner, and row indices increase downward.
- A **200 × 300** image has 200 rows and 300 columns → **60,000 pixels** in total.
- Each pixel stores one or more numeric values (intensity, or R/G/B) — this is exactly why we can feed images into mathematical models.



Pixel (row=2, col=2) highlighted

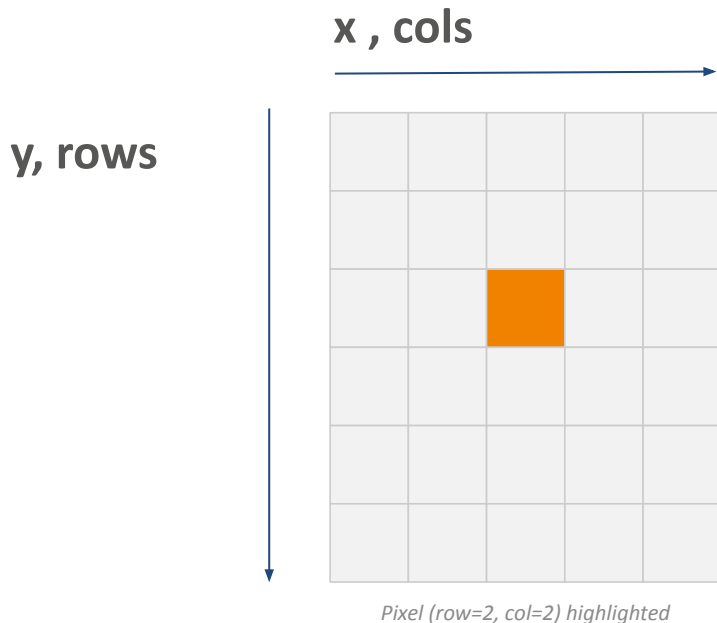
Pixel Indexing in Practice

? *img.size in PIL and arr.shape in NumPy -- are they even in the same order?*



Pixel Indexing in Practice

- ? *img.size in PIL and arr.shape in NumPy -- are they even in the same order?*
- o Mathematically we often write coordinates as **(x, y)** — horizontal, then vertical. But array libraries index as **(row, column)** — vertical, then horizontal.



Pixel Indexing in Practice

? *img.size in PIL and arr.shape in NumPy -- are they even in the same order?*

- Mathematically we often write coordinates as **(x, y)** — horizontal, then vertical. But array libraries index as **(row, column)** — vertical, then horizontal.
- This means: **array[y, x]** in NumPy corresponds to point **(x, y)** in image coordinates — the order is swapped!
- PIL's **.size** returns **(width, height)**, but **np.array(img).shape** returns **(height, width, channels)** — also swapped.
- Get in the habit of **printing shapes** to check.

CODE! → ISIC_0051817.jpg

REMEMBER

PIL size = **(W, H)** · NumPy/array shape = **(H, W, C)** · array indexing = **[row, col]** = **[y, x]**

Intensity Values & Bit Depth

- The **intensity** (or gray level) at a pixel measures how light or dark it is.
- Most images use **8-bit depth**: each pixel value ranges from **0 (black)** to **(white)** — possible levels per channel.
- Higher bit depth (e.g., 12-bit or 16-bit, common in medical/scientific imaging) gives a wider **dynamic range** — finer distinctions between similar intensities.
- 8-bit is the default for most computer-vision datasets (including **MILK10k**) because it balances **detail** and **file size**.

Intensity Values & Bit Depth

- The **intensity** (or gray level) at a pixel measures how light or dark it is.
- Most images use **8-bit depth**: each pixel value ranges from **0 (black)** to **255 (white)** — 256 possible levels per channel.
- Higher bit depth (e.g., 12-bit or 16-bit, common in medical/scientific imaging) gives a wider **dynamic range** — finer distinctions between similar intensities.
- 8-bit is the default for most computer-vision datasets (including **MILK10k**) because it balances **detail** and **file size**.

CODE!

- *Create a plot of subplots - 3 subplot*
 - *RED channel*
 - *GREEN channel*
 - *BLUE channel*
 - *Print the mean intensity of each channel*

Image Resolution

- **Spatial resolution** = how many pixels represent the image, i.e., its **width × height**.
- A color image's full shape is **Height × Width × Channels** — e.g., a 224×224 RGB image has shape **224 × 224 × 3**.
- Higher resolution → more visual detail, but also more memory and slower training.
- Deep-learning models usually **resize** all images to one fixed resolution (e.g., 224×224 for many CNNs) before training.



SESSION 2 · THEORY

Color & Intensity

RGB · grayscale · histograms

Human Color Perception

? *Your eye has only three types of color receptor. Why do screens use exactly three channels?*

Human Color Perception

- ? *Your eye has only three types of color receptor. Why do screens use exactly three channels?*
- Human color vision is **trichromatic**: the retina has three types of cone cells, each most sensitive to a different wavelength band — roughly **S** (short/blue), **M** (medium/green), **L** (long/red).
- Every color we perceive is the brain's interpretation of the **relative response** of these three cone types — not a direct measurement of wavelength.
- Digital color imaging **mimics this trichromacy directly**: cameras and screens use three channels — **Red, Green, Blue** — because that's enough to reproduce almost any color a human can see.
- This is why RGB, not some arbitrary number of channels, became the default color representation in computer vision.

RGB Color Channels

- Digital color images typically use the **RGB (Red, Green, Blue)** additive color model — mixing amounts of red, green, and blue light.
- Each channel is its own 8-bit layer (0–255); combined, a pixel can represent about **16.7 million colors** (256^3).
- An RGB image is stored as a 3D array of shape **H × W × 3** — one full grayscale-sized layer per channel, stacked together.
- Example: **pure red** = (?, ?, ?); **white** = (?, ?, ?); **black** = (?, ?, ?).

RGB Color Channels

- Digital color images typically use the **RGB (Red, Green, Blue)** additive color model — mixing amounts of red, green, and blue light.
- Each channel is its own 8-bit layer (0–255); combined, a pixel can represent about **16.7 million colors** (256^3).
- An RGB image is stored as a 3D array of shape **H × W × 3** — one full grayscale-sized layer per channel, stacked together.
- Example: **pure red** = (255, 0, 0); **white** = (255, 255, 255); **black** = (0, 0, 0).

CODE!

Create a plot fo 5 subplots of 5x5 images:

1st image: all white

2nd image: all black

3rd image: pure red

4th image: pure green

5th image: pure blue

Visualizing RGB Channels

? *If you could keep only ONE color channel, which would carry the most useful signal?*

R channel
(0–255)

G channel
(0–255)

B channel
(0–255)

- Splitting an image into its channels shows how each color contributes — e.g., skin tone and lesion-color differences (crucial in **MILK10k!**) show up differently across R, G, and B.
- Some CV pipelines process channels **separately**; deep networks usually consume all 3 **jointly**.

CODE!



TIME FOR A BREAK

20 Minutes - until 16:35h

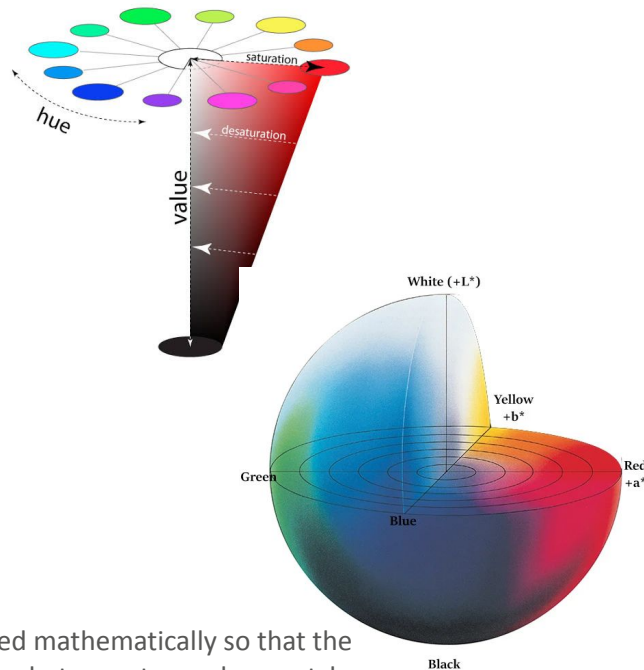
Stretch, grab a coffee, and be back on time.

Other Color Spaces — A Quick Look

- RGB is not the only way to represent color.

Color space	Channels	Why it's useful
RGB	Red, Green, Blue	matches how displays & sensors work
HSV	Hue, Saturation, Value	separates color from brightness — robust to lighting changes
Lab	Lightness (black to white), a (green to red), b (blue to yellow)	perceptually uniform — equal numeric distance $\approx$ equal perceived color difference

- Hue — the actual color itself (red, green, blue, yellow...), represented as an angle around a color wheel (0–360°)
- Saturation — how vivid/pure vs. washed-out/gray the color is
- Value — how bright or dark it is



- Lab was designed mathematically so that the numeric distance between two colors matches how different a human perceives them to be.

Grayscale Representation

- **Grayscale** collapses the 3 color channels into a **single intensity channel**, shape $H \times W \times 1$ (or just $H \times W$).
- A common conversion is the **luminosity method**:
 - **Gray = $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$** — weighted by how the human eye perceives each color's brightness (we're most sensitive to green, least to blue).
- Grayscale is useful for classical filters, edge detection, and reducing computation.
- But converting to grayscale **discards color information** — sometimes information the task actually needs.

RGB vs. Grayscale — Trade-offs

Aspect	RGB	Grayscale
Data per pixel		
File size / compute		
Color information		
Typical use		

- For **MILK10k**, color often carries diagnostic signal — converting to grayscale too early can **remove information the model needs**.

RGB vs. Grayscale — Trade-offs

Aspect	RGB	Grayscale
Data per pixel	3 values	1 value
File size / compute	3× larger	smaller, faster
Color information	preserved	lost
Typical use	most deep-learning models	classical filters, some preprocessing

- For **MILK10k**, color often carries diagnostic signal — converting to grayscale too early can **remove information the model needs**.

The Image Histogram

- A histogram



The Image Histogram

- A **histogram** counts how many pixels fall at each intensity level (0–255) — a compact summary of an image's **tonal distribution**.
- For a grayscale image, it's a single distribution; for RGB, we usually plot **one histogram per channel**.
- Histograms answer questions no single pixel can: is this image dark or bright overall? High or low contrast? Do R, G, B have similar distributions, or does one dominate?

CODE!

WHERE THIS MATTERS LATER

Histogram-based features feed **classical ML**; histogram shape flags **preprocessing issues**.

Reading a Histogram

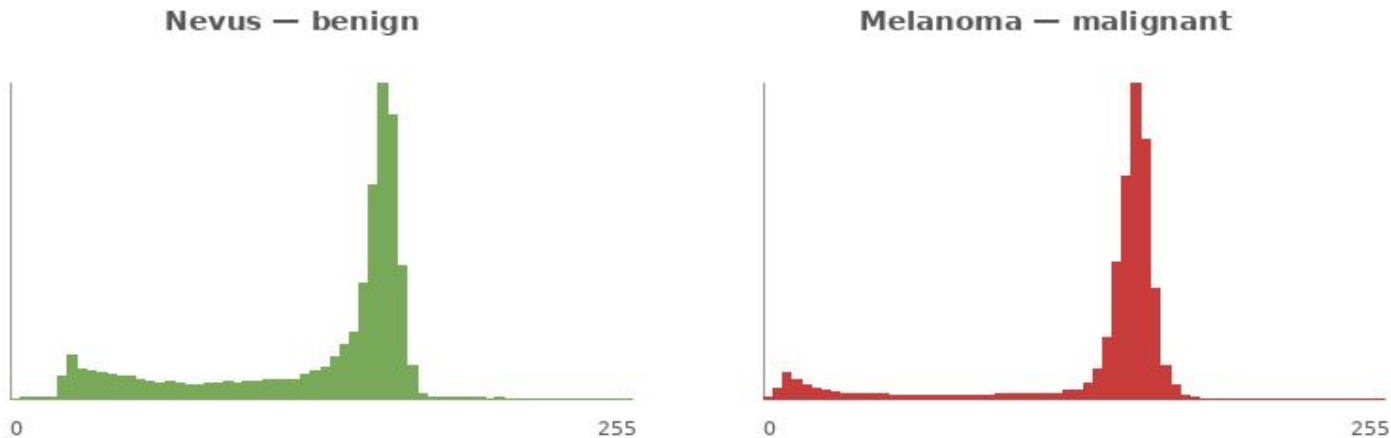
- **Mass on the left** (near 0) →
- **Mass on the right** (near 255)
- **Narrow histogram** (values clustered together) → **contrast**; **wide, spread-out histogram** → **contrast**.

Reading a Histogram

- **Mass on the left** (near 0) → many dark pixels → image is **underexposed** or has large dark regions.
- **Mass on the right** (near 255) → many bright pixels → image is **overexposed** or has large bright regions.
- **Narrow histogram** (values clustered together) → **low contrast**; **wide, spread-out histogram** → **high contrast**.

Histograms in Practice — Benign vs. Malignant

- Real grayscale intensity histograms for two actual MILK10k lesions — a benign nevus and a malignant melanoma.
- Both are dermoscopy close-ups of skin, so their overall shape is similar — but the malignant lesion's histogram carries a heavier low-intensity tail from its darker, more irregular pigmentation.



Generated from real MILK10k images (ISIC_0073863, ISIC_8416462)

Normalization of Image Data — Why It Matters

- Raw pixel values (0–255) sit on a very different scale than the small weights a neural network starts with — this can make training **slow or unstable**.
- **Normalizing** rescales pixel values to a smaller, consistent range before they enter a model.
- Benefits: faster, more stable convergence; comparable scale across channels; consistency between training and inference.
- Rule of thumb: **whatever normalization you use in training, you must use identically at inference**.

Common Normalization Techniques

Technique	Formula	When to use
Min-max scaling	$x' = x / 255$	simple default first step (0–1 range)
Standardization (z-score)	$x' = (x - \text{mean}) / \text{std}$	when a model expects zero-centered input
Dataset-specific stats	mean/std computed on your own data	best when images differ a lot from natural photos (e.g., MILK10k)
Pretrained-model stats	ImageNet mean/std ($\approx [0.485, 0.456, 0.406]$)	required for transfer learning

Part 1 Recap — Key Takeaways

- An image is a **matrix of pixels**, formally the function $F(X, Y)$; its amplitude is the pixel's **intensity**.
- **Resolution** and **bit/color depth** determine how much spatial and tonal detail an image captures; sampling/quantization are the two approximations behind them.
- **RGB** images stack three 0–255 channels into a $H \times W \times 3$ array; **grayscale** collapses them into one channel — at the cost of color information.
- **Histograms** summarize tonal distribution; **normalization** rescales pixel values so neural networks train faster and more reliably.



SESSION 2

Part 2 — Hands-on Python

From pixels to a normalized, model-ready tensor

Practice Homework

Pixels & Resolution

- Load one MILK10k image and report, in your own words:
- Its spatial resolution (height × width) — what does this number physically mean for image detail?
- Its shape as a NumPy array, and why there are 3 numbers, not 2.
- The dtype of the array, and what range of values that implies.
- Take a center crop of the image (e.g. 100×100 pixels from the middle) and display it next to the original.
In 1–2 sentences: what did you lose, and would this matter for diagnosing a lesion?

Practice Homework

Color Depth

- Simulate reduced color/intensity depth: take the image and quantize it from 8-bit (256 levels) down to 4-bit (16 levels), then 2-bit (4 levels), then 1-bit (2 levels — pure black/white). Display all four side by side.
- Question: at what bit depth does the lesion boundary become hard to distinguish from the surrounding skin? Why does this matter for a diagnostic tool?

RGB Channels

- Split an image into its R, G, B channels and display each channel as a separate grayscale image (not as red/green/blue-tinted — show the raw intensity of each channel).
- Question: which channel do you think carries the most useful signal for distinguishing a lesion from normal skin, and why? (No wrong answer here; just reason about it from what you observe.)

Grayscale Conversion

- Convert an image to grayscale two ways:
 - The naive average: $(R + G + B) / 3$
 - The luminosity-weighted formula: $0.299R + 0.587G + 0.114B$
- Display both grayscale results side by side and compute the max pixel difference between them.
Question: why do these two methods disagree, and which one better matches human perception?

Practice Homework

Histograms

- Plot the grayscale intensity histogram for one image.
- Plot the per-channel RGB histograms (R, G, B overlaid on one plot, different colors) for the same image.
- Pick two images from different diagnosis classes and plot their grayscale histograms side by side.

Question: do you see a visible difference in the histogram shape? What might explain it (lighting, lesion size, skin tone)?

Normalization

- Apply min-max normalization to an image so pixel values map from $[0, 255]$ to $[0.0, 1.0]$.
- Apply z-score standardization (subtract mean, divide by standard deviation) to the same image.
- Plot histograms of the raw, min-max normalized, and z-score standardized versions side by side (3 subplots). Question: which of these would you use as input to a neural network, and why does normalization matter for training stability? (One or two sentences — we'll cover this properly when we get to model training.)

Setup Recap

- Same environment as Session 1 — today we'll lean more heavily on **NumPy** and **OpenCV**.

```
setup.py

import numpy as np
import pandas as pd
from PIL import Image
import cv2
import matplotlib.pyplot as plt
from pathlib import Path

DATA_DIR = Path("milk10k")
df = pd.read_csv(DATA_DIR / "metadata.csv")
IMG_DIR = DATA_DIR / "images"

row = df.iloc[0]
img_path = IMG_DIR / f'{row["isic_id"]}.jpg'
```

Loading an Image as a NumPy Array

? You just loaded a JPEG with PIL. What's the one line that turns it into a matrix of numbers?

```
load_array.py

img = Image.open(img_path).convert("RGB")
arr = np.array(img)

print(type(arr))           # <class 'numpy.ndarray'>
print(arr.shape)           # (H, W, 3)  <- height, width, channels
print(arr.dtype)           # uint8
```

o Remember from theory: **PIL.size** is (W, H), but **array.shape** is (H, W, C) — always print and check when in doubt.

Inspecting Shape, dtype, and Value Range

```
inspect.py

print("shape:", arr.shape)
print("dtype:", arr.dtype)
print("min / max:", arr.min(), arr.max())
print("mean per channel:", arr.reshape(-1, 3).mean(axis=0))
```

- **arr.reshape(-1, 3)** flattens the H×W grid into a list of pixels, so **.mean(axis=0)** gives the average R, G, B separately — your first per-channel statistic, and a preview of what normalization needs.

Indexing and Slicing Pixels

```
slicing.py

# a single pixel at row=50, col=100 -> array[row, col]
pixel = arr[50, 100]
print(pixel)                # e.g. [188 142 130]  (R, G, B)

# crop a 100x100 region (rows 0-99, cols 0-99)
crop = arr[0:100, 0:100]
print(crop.shape)           # (100, 100, 3)

plt.imshow(crop)
plt.axis("off")
plt.show()
```

- NumPy slicing is **[rows, columns]** — exactly the (H, W) order from theory.

Splitting and Visualizing RGB Channels

```
split_channels.py
```

```
R, G, B = arr[:, :, 0], arr[:, :, 1], arr[:, :, 2]
```

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
```

```
for ax, channel, name in zip(axes, [R, G, B], ["Red", "Green", "Blue"]):
```

```
    ax.imshow(channel, cmap="gray")
```

```
    ax.set_title(name)
```

```
    ax.axis("off")
```

```
plt.show()
```

- `arr[:, :, 0]` takes every row, every column, only channel 0 (Red) — this is exactly the $H \times W \times 3$ shape from theory, sliced along the last axis.

Converting to Grayscale — Two Ways

? *Do you expect a hand-written formula to match OpenCV's built-in grayscale conversion exactly?*

```
● ● ● grayscale.py  
  
# Method 1: manual luminosity formula (from theory)  
gray_manual = (0.299 * R + 0.587 * G + 0.114 * B).astype(np.uint8)  
  
# Method 2: OpenCV's built-in conversion  
gray_cv2 = cv2.cvtColor(arr, cv2.COLOR_RGB2GRAY)  
  
print(np.allclose(gray_manual, gray_cv2, atol=1)) # True — same formula!
```

- o OpenCV's **cv2.cvtColor** uses essentially the same luminosity weights — good confirmation that the formula from theory is exactly what production libraries use.

Plotting an Intensity Histogram

? *What's the one line of code that turns a 2D image into a histogram?*

```
histogram.py  
  
plt.figure(figsize=(8, 4))  
plt.hist(gray_manual.ravel(), bins=256, range=(0, 255), color="#555555")  
plt.xlabel("Pixel intensity")  
plt.ylabel("Pixel count")  
plt.title("Grayscale intensity histogram")  
plt.show()
```

- `.ravel()` flattens the 2D image into a 1D list of intensity values so **plt.hist** can bin and count them.
- Use the reading guide from theory: where is the mass concentrated? Is the image high- or low-contrast?

Comparing Histograms Across Classes

```
compare_histograms.py

def gray_of(isic_id):
    img = Image.open(IMG_DIR / f"{isic_id}.jpg").convert("RGB")
    return cv2.cvtColor(np.array(img), cv2.COLOR_RGB2GRAY)

sample = df.groupby("diagnosis_1").sample(1, random_state=0)

fig, axes = plt.subplots(1, len(sample), figsize=(4 * len(sample), 3))
for ax, (_, row) in zip(axes, sample.iterrows()):
    g = gray_of(row["isic_id"])
    ax.hist(g.ravel(), bins=64, range=(0, 255))
    ax.set_title(row["diagnosis_1"], fontsize=9)
plt.tight_layout()
plt.show()
```

df.groupby("diagnosis_1").sample(1) grabs one example per class — a fast way to eyeball differences across categories.

Min-Max Normalization in Code

```
minmax.py

arr_float = arr.astype(np.float32)
normalized = arr_float / 255.0

print(normalized.dtype)          # float32
print(normalized.min(), normalized.max())  # 0.0 1.0
```

- Cast to **float32 first** — dividing uint8 by 255 without casting can silently produce all-zero results in some libraries.

Standardization (Z-score) in Code

```
standardize.py

# per-channel mean/std over a sample of MILK10k images
sample = df.sample(200, random_state=0)
paths = [IMG_DIR / f'{r["isic_id"]}.jpg' for _, r in sample.iterrows()]
pixels = np.concatenate(
    [np.array(Image.open(p).convert("RGB")).reshape(-1, 3) for p in paths]
)

mean, std = pixels.mean(axis=0) / 255.0, pixels.std(axis=0) / 255.0
print("dataset mean:", mean, " std:", std)

standardized = (normalized - mean) / std
```

This is exactly the kind of dataset-specific statistic Session 3 will compute properly (on the train split only, to avoid leakage).

Using ImageNet Pretrained Stats

? Should new images be normalized with MILK10k's stats, or ImageNet's?

torchvision_transform.py

```
import torchvision.transforms as T

preprocess = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),           # HWC [0,255] uint8 -> CHW [0,1] float
    T.Normalize(mean=[0.485, 0.456, 0.406],
                 std=[0.229, 0.224, 0.225]), # ImageNet stats
])

tensor = preprocess(img)
print(tensor.shape)      # torch.Size([3, 224, 224])
```

- When we fine-tune an **ImageNet-pretrained CNN**, we must normalize with **its** statistics — not MILK10k's — so inputs match what the pretrained weights expect.

Building a Preprocessing Function

? *Resize, normalize, reorder channels -- does the order of those three steps actually matter?*

```
preprocess.py

def preprocess_image(path, size=(224, 224), mean=None, std=None):
    img = Image.open(path).convert("RGB").resize(size)
    arr = np.array(img).astype(np.float32) / 255.0           # 1. min-max

    if mean is not None and std is not None:
        arr = (arr - mean) / std                             # 2. standardize

    tensor = np.transpose(arr, (2, 0, 1))                   # 3. HWC -> CHW
    return tensor

x = preprocess_image(img_path, mean=mean, std=std)
print(x.shape, x.dtype)    # (3, 224, 224) float32
```

load → resize → normalize → channels-first tensor: the exact pipeline every model in this course will call before training.

Coming Up Next & References

- **Next — Session 3: Data Preparation for Machine Learning Projects.** Train/validation/test splits, data leakage, class imbalance, data augmentation, and building robust data pipelines — all applied to **MILK10k**.

- **References & further reading**

- Hugging Face — Computer Vision Course, Unit 1: huggingface.co/learn/computer-vision-course/unit1/chapter1/definition
- MIT Visionbook — Color chapter: visionbook.mit.edu/color.html